

Competitive Programming

Section 51 Editorial: Invariants In Hard Problems

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. Same Parity	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
3	B. XOR Balanced Splits	5
3.1	Restatement	5
3.2	Observations and algorithm	5
3.3	Correctness proof	5
3.4	Complexity	5
3.5	Full C++ solution	5
3.6	Full Python solution	5
4	C. Same Residue	7
4.1	Restatement	7
4.2	Observations and algorithm	7
4.3	Correctness proof	7
4.4	Complexity	7
4.5	Full C++ solution	7
4.6	Full Python solution	7
5	What to carry forward	8

1 Editorial overview

1.1 Detailed solution guide

A. Parity Invariant. Each allowed operation changes the state but preserves the parity quantity named in the statement. Therefore states with different parity are unreachable from each other. To prove sufficiency, show a constructive sequence that reaches a canonical state for every configuration with the same parity. The answer is then determined entirely by that invariant.

B. Prefix XOR Splits. XOR is its own inverse, so the XOR of subarray $[l, r]$ is $\text{pref}[r] \text{ xor } \text{pref}[l-1]$. A split condition can therefore be checked by comparing prefix values rather than recomputing ranges. The algorithm scans the array while maintaining the prefix values that make the remaining segments possible.

C. Modulo Residue Invariant. If every operation changes a value by a multiple of m , then the residue modulo m is fixed. Conversely, if two values have the same residue, their difference is divisible by m , so the operation can bridge the gap in steps of size m . This gives both the necessary and the sufficient direction.

Common pitfalls. An invariant proof has two halves: impossibility for different invariant values and reachability or optimality within the same invariant class.

The solutions in this section are intentionally short because the hard work is the invariant.

- A uses parity: changing a value by 2 preserves parity, and equal parity is also sufficient.
- B uses prefix xor. A split has equal left and right xor exactly when the prefix xor matches the suffix xor.
- C uses residues modulo k : operations changing a number by multiples of k cannot alter its residue.

Each proof has both halves: the invariant is necessary because operations preserve it, and sufficient because a direct transformation or count follows once the invariant condition holds.

2 A. Same Parity

2.1 Restatement

Can all numbers become equal if one operation changes a number by 2?

2.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

2.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. \square

2.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

2.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    a=data[1:]; print("YES" if all(x%2==a[0]%2 for x in a) else "NO")
if __name__=="__main__": main()
```

3 B. XOR Balanced Splits

3.1 Restatement

Count split positions where left xor equals right xor.

3.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

3.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

3.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

3.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    a=data[1:]; total=0
    for x in a: total^=x
```

```
pref=ans=0
for x in a[:-1]:
    pref^=x
    if pref==(total^pref): ans+=1
print(ans)
if __name__=="__main__": main()
```

4 C. Same Residue

4.1 Restatement

Can all numbers become equal using operations that change a number by k ?

4.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

4.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

4.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;long long
```

4.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,k=data[0],data[1]; a=data[2:]; print("YES" if len({x%k for x in a})==1 else "NO")
if __name__=="__main__": main()
```

5 What to carry forward

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.